OOP in Python with Examples

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** and **classes** rather than functions and logic alone. It focuses on creating reusable, modular, and scalable code by modeling real-world entities.

In OOP, data (attributes) and behavior (methods) are bundled together into **classes**, which act as blueprints for creating **objects**. These objects interact with each other to perform various tasks, similar to how real-world entities interact.

OOP

- Promotes **code reusability** through inheritance and composition.
- Increases **modularity** — each class has a specific responsibility.
- Simplifies **maintenance and debugging** by isolating functionality.
- Enables **data encapsulation**, protecting sensitive information.
- Supports **polymorphism**, allowing flexible and dynamic code behavior.

Python supports OOP fully, and most real-world Python applications (like web frameworks, ML pipelines, and GUIs) use OOP principles extensively.

1. Class and Object

A **class** is a blueprint or template that defines the structure and behaviour (attributes and methods) of objects. An **object** is an instance of a class that holds actual data. Classes are used to represent real-world entities like cars, users, or bank accounts in code. Objects created from these classes carry unique data while sharing the same structure and functionality.

Example

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner   # Instance variable for account holder's name
        self.balance = balance   # Instance variable for account balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited ₹{amount}. New balance: ₹{self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew ₹{amount}. Rem. balance: ₹{self.balance}")
        else:
            print("Insufficient balance!")

# --- Usage ---
acc1 = BankAccount("Alice", 5000)
acc2 = BankAccount("Bob", 2000)

acc1.deposit(1500)
acc2.withdraw(1000)
```

Explanation:

- The **BankAccount class** defines the structure and behavior for all accounts (attributes: owner, balance; methods: deposit, withdraw).
- **Objects** like acc1 and acc2 represent *individual customers* with their own data.
- Each object works independently — Alice's transactions don't affect Bob's.

Note:
A class defines the **blueprint**, while objects are **real-world instances** that hold unique data and execute the behaviour defined by the class.

2. Constructor (init)
__init__ is a special method called automatically when a new object is instantiated. It initializes instance variables and can enforce constraints or perform setup logic.

Example:

```
class Customer:
    def __init__(self, name, acc_type):
    """
    Default constructor called automatically when a new object is created.
        Initializes instance variables and can perform setup logic.
    """
        self.name = name
        self.acc_type = acc_type
        print(f"New cust. created: {self.name}, {self.acc_type} acc.")

# Alt constructor: create a Cust with only name, defaulting acc_type
    @classmethod
    def from_name(cls, name):
        print("Creating customer using from_name() constructor...")
        return cls(name, "Savings")

    # Another alternative constructor: create a Customer from a dictionary
    @classmethod
    def from_dict(cls, data):
        print("Creating customer using from_dict() constructor...")
        return cls(data.get("name"), data.get("account_type", "Checking"))

# --- Usage examples ---

# Normal object creation (calls __init__ directly)
c1 = Customer("Alice", "Premium")

# Alternative constructor using classmethod
c2 = Customer.from_name("Bob")

# Alternative constructor using dictionary input
customer_data = {"name": "Charlie", "account_type": "Gold"}
c3 = Customer.from_dict(customer_data)
```

Note:
Unlike other languages, Python allows multiple constructors via @classmethod alternative

constructors. They enable flexible ways to instantiate objects from different data sources (like dictionaries, files, or API responses) while maintaining clean and consistent object creation logic.

3. Attributes and Methods

- **Instance attributes:** Stored per-object (`self.attribute`). Each object gets its own copy of the data.
- **Class attributes:** Shared across all instances (`ClassName.attribute`). Useful for constants or shared data.
- **Instance methods:** Operate on specific object data, accessing instance attributes through `self`.

Example:

```
class Bank:
    bank_name = "National Bank"  # Class var (shared by all instances)

    def __init__(self, acc_holder):
        self.acc_holder = acc_holder  # Instance var (unique to each obj.)

    def welcome(self):
        print(f"Welcome {self.acc_holder} to {Bank.bank_name}!")

# --- Usage examples ---
# Creating two objects (each with its own instance attribute)
customer1 = Bank("Alice")
customer2 = Bank("Bob")
customer1.welcome()  # Output: Welcome Alice to National Bank!
customer2.welcome()  # Output: Welcome Bob to National Bank!

# Changing the class variable affects all instances
Bank.bank_name = "Global Trust Bank"
customer1.welcome()  # Output: Welcome Alice to Global Trust Bank!
customer2.welcome()  # Output: Welcome Bob to Global Trust Bank!
```

- Each instance's attributes are stored in its internal `__dict__`.
- When you access an attribute (e.g., `self.bank_name`), Python first checks the instance's `__dict__`. If it's not found, it looks up the class and parent classes — this is the **class namespace resolution chain**.

Note:
Use **instance attributes** for data unique to each object and **class attributes** for values common to all objects. This design keeps memory efficient and behavior consistent across instances.

4. Encapsulation
Encapsulation is the principle of restricting direct access to an object's internal state and requiring all interactions to occur through methods. It helps protect data integrity and hide implementation details.

Python Access Levels:

Python uses naming conventions to indicate the intended level of access:

- **Public:** Accessible anywhere (self.name)
- **Protected:** Intended for internal use only (self._name)
- **Private:** Name-mangled to prevent accidental access (self.__name → _ClassName__name)

Example:

```
class Account:

    def __init__(self, owner, balance):
        self.owner = owner              # Public attribute
        self._status = "Active"         # Protected attribute)
        self.__balance = balance        # Private attribute (name-mangled)

    def deposit(self, amount):
            if amount > 0:
                self.__balance += amount
                print(f"Deposited {amount}. New balance:
{self.__balance}")
            else:
                print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew {amount}. Remaining balance:
{self.__balance}")
        else:
            print("Invalid withdrawal amount.")

    def get_balance(self):
        # Controlled access to private attribute
        return self.__balance

# --- Usage examples ---
acc = Account("Alice", 500)
acc.deposit(200)             # Works
acc.withdraw(100)            # Works
print(acc.get_balance())     # Access balance safely

# Trying to access the private attribute directly
# print(acc.__balance)       # AttributeError: 'Account' object has no
attribute '__balance'

# But name mangling still allows access (not recommended)
print(acc._Account__balance)  # Works, but breaks encapsulation
```

Note:
Encapsulation maintains **data integrity** by ensuring sensitive data (like balance) cannot be directly modified or corrupted. Instead, access and updates occur through controlled methods (deposit, withdraw, get_balance).

5. Inheritance

Inheritance allows a subclass to derive attributes and behaviors from a parent class, promoting **code reuse**, **hierarchical organization**, and **extensibility**.

A **base class (parent)** defines common attributes and methods, while a **derived class (child)** extends or customizes them.

Types of Inheritance in Python:

1. **Single Inheritance** — One parent, one child.
2. **Multilevel Inheritance** — Child inherits from a parent, and another class inherits from that child.
3. **Multiple Inheritance** — A child class inherits from more than one parent.
4. **Hierarchical Inheritance** — Multiple child classes share the same parent.
5. **Hybrid Inheritance** — A mix of the above types.

Single Inheritance

One parent → One child

```
       Tree
        |
     MangoTree
```

```
# Parent class
class Tree:
    def create(self):
        print("Parent Tree is created")


# Child class
class MangoTree(Tree):
    def mangoCreate(self):
        print("Mangoes tree created")


# Object
tree = MangoTree()

tree.create()          # Inherited method
tree.mangoCreate()     # Child method
```

Output:

```
Parent Tree is created
Mangoes tree is created
```

Here, `MangoTree` inherits from `Tree`.

Example 1 — Single Inheritance

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount}. Remaining balance: {self.balance}")
        else:
            print("Invalid withdrawal amount.")

# Derived class (inherits from Account)
class SavingsAccount(Account):
    def __init__(self, owner, balance, interest_rate):
        # Initialize parent attributes using super()
        super().__init__(owner, balance)
        self.interest_rate = interest_rate
    def add_interest(self):
        interest = self.balance * self.interest_rate / 100
        self.balance += interest
        print(f"Interest of {interest} added. New balance:
{self.balance}")

# --- Usage ---
acc = SavingsAccount("Alice", 1000, 5)
acc.deposit(500)
acc.add_interest()    # Inherited + extended behavior
acc.withdraw(200)
```

- SavingsAccount inherits all attributes and methods from Account.
- The super().__init__() call ensures that the parent constructor runs properly.
- New method add_interest() is specific to the subclass.

Multi-Level Inheritance

Grandparent → parent → child

```
          Tree
            |
        FruitTree
            |
      MangoTree

# Grandparent class
```

```
class Tree:
    def grow(self):
        print("Tree is growing")


# Parent class
class FruitTree(Tree):
    def produce_fruit(self):
        print("Fruit tree produces fruit")


# Child class
class MangoTree(FruitTree):
    def show_fruit(self):
        print("Mango tree produces mangoes")


# Object
tree = MangoTree()

tree.grow()           # From Tree
tree.produce_fruit()  # From FruitTree
tree.show_fruit()     # From MangoTree
```

Output:

```
Tree is growing
Fruit tree produces fruit
Mango tree produces mangoes
```

Here, `MangoTree` can access methods from **both `FruitTree` and `Tree`**.

Example 2 — Multilevel Inheritance:

```
class PremiumSavings(SavingsAccount):

    def __init__(self, owner, balance, interest_rate, bonus_rate):
        super().__init__(owner, balance, interest_rate)
        self.bonus_rate = bonus_rate

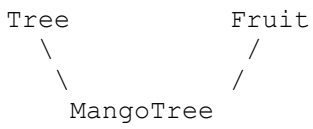
    def add_bonus(self):
        bonus = self.balance * self.bonus_rate / 100
        self.balance += bonus
        print(f"Bonus of {bonus} added. New balance: {self.balance}")

# --- Usage ---
premium = PremiumSavings("Bob", 2000, 4, 2)
premium.add_interest()  # From SavingsAccount
premium.add_bonus()      # From PremiumSavings
```

Here, PremiumSavings inherits from SavingsAccount, which itself inherits from Account — forming a **multilevel inheritance chain**.

Multiple Inheritance

Multiple parents → one child




```
# Parent class 1
class Tree:
    def grow(self):
        print("Tree is growing")


# Parent class 2
class Fruit:
    def produce_fruit(self):
        print("Tree produces fruit")


# Child class
class MangoTree(Tree, Fruit):
    def show(self):
        print("This is a mango tree")


# Object
tree = MangoTree()

tree.grow()             # From Tree
tree.produce_fruit()    # From Fruit
tree.show()             # Own method
```

Output:

```
Tree is growing
Tree produces fruit
This is a mango tree
```

Here, MangoTree inherits from **two parent classes**: Tree and Fruit.

Example 3 — Multiple Inheritance:

```
class Rewards:
    def redeem_points(self):
        print("Redeeming reward points...")

class RewardedAccount(SavingsAccount, Rewards):
    def __init__(self, owner, balance, interest_rate, points):
        SavingsAccount.__init__(self, owner, balance, interest_rate)
        self.points = points
    def show_points(self):
        print(f"You have {self.points} reward points.")
```

```
# --- Usage ---
reward_acc = RewardedAccount("Clara", 1500, 3, 120)
reward_acc.add_interest()     # From SavingsAccount
reward_acc.redeem_points()    # From Rewards
reward_acc.show_points()      # Defined in RewardedAccount
```

In **multiple inheritance**, the class inherits methods from both parent classes (SavingsAccount and Rewards).

Technical Detail:

- The super() function follows **MRO (Method Resolution Order)** for deterministic inheritance.
- Python's **C3 Linearization** ensures that all parent classes are initialized in a consistent, predictable order.

You can view MRO using:

```
print(RewardedAccount.mro())
```

This prints the inheritance hierarchy order Python follows while resolving methods.

Note:
Inheritance promotes **reusability** and **extension** of existing code, reducing redundancy. However, it should be used judiciously — deep hierarchies can make debugging complex.

Example: Inheritance with a Tree Structure

Think of a tree where:

- `Tree` is the **parent class**
- `FruitTree` and `FlowerTree` are **child classes**
- Child classes inherit properties and methods from `Tree`

```
# Parent class
class Tree:
    def __init__(self, name, height):
        self.name = name
        self.height = height

    def show_info(self):
        print("Tree Name:", self.name)
        print("Height:", self.height, "meters")


# Child class 1
class FruitTree(Tree):
    def __init__(self, name, height, fruit):
        super().__init__(name, height)
        self.fruit = fruit

    def show_fruit(self):
```

```
        print("Fruit:", self.fruit)


# Child class 2
class FlowerTree(Tree):
    def __init__(self, name, height, flower):
        super().__init__(name, height)
        self.flower = flower

    def show_flower(self):
        print("Flower:", self.flower)


# Creating objects
mango = FruitTree("Mango Tree", 10, "Mango")
gulmohar = FlowerTree("Gulmohar Tree", 8, "Red Flower")


# Using inherited method
mango.show_info()
mango.show_fruit()

print()

gulmohar.show_info()
gulmohar.show_flower()
```

Output

```
Tree Name: Mango Tree
Height: 10 meters
Fruit: Mango

Tree Name: Gulmohar Tree
Height: 8 meters
Flower: Red Flower
```

6. Polymorphism

Polymorphism allows a single interface to represent multiple underlying forms. It enables flexibility and scalability in object-oriented design by allowing objects of different classes to respond to the same method call in their own way.

a) Method Overriding (Runtime Polymorphism)

Method overriding occurs when a subclass provides its own implementation of a method already defined in its parent class. The version that gets executed is determined **at runtime**, depending on the object type.

Example:

```
class Account:
    def account_type(self):
        print("General Account")

class SavingsAccount(Account):
    def account_type(self):
        print("Savings Account")

# --- Usage ---
acc1 = Account()
acc2 = SavingsAccount()
acc1.account_type()    # Output: General Account
acc2.account_type()    # Output: Savings Account
```

- Both Account and SavingsAccount have the same method account_type().
- When called on a SavingsAccount object, the overridden method runs.
- This behavior is **runtime polymorphism** — the method executed depends on the **actual object type**, not the reference type.

b) Method Overloading (Compile-time Simulation)
Python does not natively support true method overloading like Java or C++. Instead, we can simulate it using **default arguments**, **variable-length arguments**, or **argument inspection**.

Example:

```
def withdraw(amount, limit=None):
    if limit:
        print(f"Withdrawing {amount} with limit {limit}")
    else:
        print(f"Withdrawing {amount}")

# --- Usage ---
withdraw(500)
withdraw(500, 1000)
```

- By giving limit a default value (None), the same function behaves differently based on how it's called.
- This is a **form of compile-time polymorphism simulation** achieved dynamically through argument inspection.

Note:

- **Method Overriding** → Achieved through inheritance; actual method chosen at runtime.
- **Method Overloading (Simulation)** → Achieved through default parameters.

Example using argument inspection:

```
def process(*args, **kwargs):
    if len(args) == 1:
        print(f"Single argument: {args[0]}")
    elif len(args) == 2:
        print(f"Two arguments: {args[0]}, {args[1]}")
    else:
        print("Unsupported number of arguments.")

process(10)
process(10, 20)
```

Note:
Polymorphism enhances **code flexibility**, **extensibility**, and **readability** — you can define a uniform interface while allowing each subclass or function variant to adapt its own behavior.

7. Abstraction
Abstraction is the process of hiding complex implementation details and exposing only the **essential functionality**. It simplifies interaction with complex systems by focusing on **what** an object does rather than **how** it does it.
In Python, abstraction is implemented using the `abc` module, which provides the `ABC` (Abstract Base Class) and `@abstractmethod` decorators.

Example:

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def make_payment(self, amount):
        pass

class CreditCardPayment(Payment):
    def make_payment(self, amount):
        print(f"Paid {amount} using Credit Card")

class UpiPayment(Payment):
    def make_payment(self, amount):
        print(f"Paid {amount} via UPI")

# --- Usage ---
# payment = Payment()  # Error: Can't instantiate abstract class
credit = CreditCardPayment()
upi = UpiPayment()
credit.make_payment(1000)    # Paid 1000 using Credit Card
upi.make_payment(500)        # Paid 500 via UPI
```

- Payment is an **abstract base class** that defines the interface (make_payment).
- Subclasses like CreditCardPayment and UpiPayment must **implement** this method.
- Trying to instantiate Payment directly raises an error because it contains unimplemented abstract methods.
- The abc.ABC class marks a class as abstract.
- @abstractmethod ensures that subclasses must override the method.
- Abstract classes enforce **contract-driven development** — any subclass must adhere to the defined interface.
- This supports **extensibility** while maintaining **design consistency**.

Note:
Abstraction ensures developers interact only with high-level methods and not the intricate internal workings. It reduces complexity, enhances maintainability, and enables uniform interfaces.

Use case: ATM
Think of using an ATM: you press buttons to withdraw money without knowing how the internal banking system, verification, or cash dispensing mechanism works. You see the **interface**, not the **implementation**.

self **and** cls **Keywords**
In Python's object-oriented programming, self and cls are two special references used to distinguish **instance-level** and **class-level** operations.

- **self** → Refers to the **instance** of the class (object) that calls the method.
- **cls** → Refers to the **class itself**, used primarily in **class methods**.

Example:

```
class Bank:
    bank_name = "Global Bank"  # Class variable shared by all instances

    def __init__(self, name):
        self.name = name  # Instance variable unique to each object

    def show_account(self):
        print(f"Customer: {self.name} | Bank: {self.bank_name}")

    @classmethod
    def change_bank(cls, new_name):
        cls.bank_name = new_name  # Modify class-level attribute

    @staticmethod
    def info():
        print("Static utility method: Bank info")

# --- Usage ---
# Instance methods use 'self'
acc1 = Bank("Alice")
acc2 = Bank("Bob")
acc1.show_account()  # Customer: Alice | Bank: Global Bank
acc2.show_account()  # Customer: Bob | Bank: Global Bank
```

```
# Class method uses 'cls' to modify shared class variable
Bank.change_bank("National Bank")
acc1.show_account()  # Customer: Alice | Bank: National Bank
acc2.show_account()  # Customer: Bob | Bank: National Bank

# Static method - no self or cls
Bank.info()  # Static utility method: Bank info
```

- **self** gives access to the current **object instance** — it's how instance variables and methods are referenced.
- **cls** gives access to the **class** — useful for class-wide updates and configurations.
- **@staticmethod** defines a method independent of both class and instance contexts; it's used for utility functions that logically belong to the class's namespace.
- self → instance context, supports **dynamic binding** of methods to objects.
- cls → class-level reference, allows modification of class attributes.
- @staticmethod → belongs to the class **namespace only**; no automatic binding.

Note:
Using self, cls, and @staticmethod properly separates **object behavior**, **class-wide configuration**, and **utility logic**, resulting in cleaner, modular code.

8. Class vs Instance Variables

In Python, **class variables** are shared across all instances of a class, while **instance variables** are unique to each object. Understanding the difference ensures predictable behavior and proper data encapsulation.

Example:

```
class Employee:
    company = "TechCorp"  # Class variable shared by all employees

    def __init__(self, name):
        self.name = name          # Instance variable unique to each
employee

    def show_info(self):
        print(f"Employee: {self.name} | Company: {self.company}")

# --- Usage ---
e1 = Employee("Alice")
e2 = Employee("Bob")

# Accessing class variable
print(Employee.company)   # Output: TechCorp

# Accessing via instances
e1.show_info()            # Employee: Alice | Company: TechCorp
e2.show_info()            # Employee: Bob | Company: TechCorp

# Modifying class variable through the class
Employee.company = "InnoTech"
print(Employee.company)    # Output: InnoTech
e1.show_info()            # Employee: Alice | Company: InnoTech
```

```
e2.show_info()              # Employee: Bob | Company: InnoTech

# Creating an instance variable with the same name (shadows class
variable)
e1.company = "StartUpX"
print(e1.company)           # Output: StartUpX (instance-specific)
print(e2.company)           # Output: InnoTech (still uses class variable)
```

- company defined inside the class is a **class variable**, shared across all instances.
- name is an **instance variable**, unique to each object.
- When an instance variable has the same name as a class variable, it **shadows** the class variable for that specific object.
- Class variables are stored in the **class namespace**.
- Instance variables are stored in the **instance's __dict__**.
- The lookup follows this order:

Note: Use **class variables** for data common to all objects (e.g., company name, configuration), and **instance variables** for object-specific data (e.g., employee name, salary).

9. Dunder (Magic) Methods

Dunder methods (short for **double underscore methods**) — also known as **magic methods** — provide a way to customize how Python objects behave with built-in operations. They enable classes to implement operator overloading and support Pythonic conventions like +, ==, len(), print(), and more.

Common Dunder Methods and Their Purposes:

1. __init__ Constructor — initializes an object's state
2. __str__ Returns a human-readable string representation (for users)
3. __repr__ Returns a developer-friendly string (for debugging)
4. __add__ Defines behavior for the + operator
5. __eq__ Defines equality comparison using ==
6. __len__ Defines behavior for the built-in len() function
7. __getitem__ Defines behavior for indexing (obj[key])
8. __call__ Allows instances to be called like functions

Example:

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __eq__(self, other):
        return self.amount == other.amount

    def __repr__(self):
        return f"Money({self.amount})"

    def __str__(self):
        return f"${self.amount}"
```

```
# --- Usage ---
m1 = Money(100)
m2 = Money(200)
m3 = m1 + m2           # Calls __add__
print(m3)               # Calls __str__ → $300
print(repr(m3))         # Calls __repr__ → Money(300)
print(m1 == Money(100))  # Calls __eq__ → True
```

- __init__ initializes the object with a given amount.
- __add__ defines custom addition, returning a new Money object.
- __eq__ allows equality comparison between objects.
- __repr__ and __str__ control how the object is displayed — one for developers (repr), the other for users (str).
- Dunder methods enhance **readability** and **integration** with Python syntax.
- They support **operator overloading**, allowing custom behavior for arithmetic, comparison, and container operations.
- Implementing them correctly makes user-defined classes behave like native types.

Note:
By implementing dunder methods, you make your objects feel like first-class Python citizens — they interact naturally with built-in operators and functions.

Operator Overloading
Operator overloading allows developers to redefine how built-in operators (like +, <, >, ==, etc.) behave for custom objects. It provides a natural, domain-specific way to perform arithmetic, comparison, and other operations directly on user-defined class instances.
By implementing **dunder (magic) methods**, you can make objects behave intuitively with standard Python operators.

Example:

```
class Account:
    def __init__(self, balance):
        self.balance = balance

    def __lt__(self, other):
        return self.balance < other.balance   # Defines < behavior

    def __add__(self, other):
        return Account(self.balance + other.balance)  # Defines + behavior

    def __eq__(self, other):
        return self.balance == other.balance   # Defines == behavior

    def __str__(self):
        return f"Balance: {self.balance}"

# --- Usage ---
acc1 = Account(500)
acc2 = Account(800)
acc3 = Account(300)
```

```
# Comparison using < (calls __lt__)
print(acc1 < acc2)   # True

# Addition using + (calls __add__)
combined = acc1 + acc3
print(combined)      # Balance: 800

# Equality using == (calls __eq__)
print(acc1 == acc3)  # False
```

- `__lt__` redefines the `<` operator to compare balances.
- `__add__` allows account balances to be summed directly using `+`.
- `__eq__` enables equality checking using `==`.
- `__str__` customizes how the object is printed, returning a human-readable form.
- Operator overloading uses **special dunder methods** like `__add__`, `__sub__`, `__mul__`, `__lt__`, `__eq__`, and others to map Python operators to custom behaviors.
- This enables **domain-specific arithmetic and comparisons**, letting objects model real-world logic naturally.
- It also improves **readability** and **maintainability**, allowing expressions like `acc1 + acc2` instead of verbose function calls.

Note:
Operator overloading makes custom objects behave like native types, allowing for **intuitive arithmetic and logical expressions** within your domain model.

10. Composition (HAS-A Relationship)
Composition is an object-oriented design principle where one class **contains** an instance of another class. It represents a **HAS-A relationship**, promoting **modularity**, **reusability**, and **flexibility** in design.
Rather than forming deep inheritance hierarchies, composition allows classes to collaborate by combining their behaviors.

Example:

```
class Customer:
    def __init__(self, name):
        self.name = name

class BankAccount:
    def __init__(self, customer, balance=0):
        self.customer = customer   # Composition: BankAccount HAS-A
Customer
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
```

```
            self.balance -= amount
            print(f"Withdrew {amount}. Remaining balance: {self.balance}")
        else:
            print("Invalid withdrawal amount.")

# --- Usage ---
c = Customer("Alice")
acc = BankAccount(c, 10000)
print(f"{acc.customer.name}'s balance: {acc.balance}")

acc.deposit(500)
acc.withdraw(2000)
```

- The BankAccount class **contains** a Customer object.
- This represents a **HAS-A relationship** — a BankAccount *has a* Customer.
- Unlike inheritance (IS-A relationship), composition models collaboration between independent objects.
- Composition provides **looser coupling** between classes.
- It avoids **tight dependencies** common in deep inheritance chains.
- Components can be easily replaced, extended, or tested independently.

Note:
Composition encourages **modular architecture**, where smaller, independent classes combine to build complex systems. It is often preferred over inheritance for cleaner and more adaptable designs.

Source: https://medium.com/@vamsikd219/object-oriented-programming-in-python-25de0ef9fd7c